



华中科技大学

计算机系统结构实验报告

姓 名: Losyi
学 院: 计算机科学与技术
专 业: 计算机科学与技术
班 级: CS2300
学 号: U202310000

分数	
教师签名	

2026 年. 5 月. 5 日

目 录

1. Cache 模拟器实验	1
1.1. 实验目的	1
1.2. 实验环境	1
1.3. 实验思路	1
1.4. 实验结果和分析	6
2. 矩阵转置实验	7
2.1. 实验目的	7
2.2. 实验环境	7
2.3. 实验思路	7
2.4. 实验结果和分析	10
3. 总结和体会	12
4. 对实验课程的建议	12

1. Cache 模拟器实验

1.1. 实验目的

本次实验的核心目的是通过编写 Cache 模拟器，深入理解计算机存储系统中高速缓存 Cache 的工作机制，将理论知识与实际编程相结合。在实验过程中，需要理解存储器层次结构中 Cache 的核心作用以及其对计算机系统性能的重要影响，熟练掌握 Cache 的基本结构，重点吃透组相联映射方式的原理与应用，同时掌握内存地址的划分方法，能够准确计算地址中的 Tag、Set Index 和 Block Offset。

除此之外，还需深入理解 LRU（最近最少使用）替换策略的核心逻辑，并能通过编程将其落地实现，能够根据给定的内存访问轨迹，准确模拟 Cache 的命中（hit）、缺失（miss）及驱逐（eviction）三种行为。同时，通过本次实验提升自身使用 C 语言进行系统级编程的能力，重点锻炼文件处理、动态内存分配和命令行参数解析等实用技能，确保模拟器能够正常运行并达到实验要求。

1.2. 实验环境

本次实验全程在头歌远程实验环境中开展，实验环境的具体配置详细如下：

操作系统：Debian GNU/Linux 9 (stretch)

内核版本：Linux 5.10.134

架构：x86_64

CPU：Intel Xeon Platinum 8269CY @ 2.50GHz（8 核 16 线程）

内存：总容量 30GB，可用容量 24GB

Swap：无

代码路径：/data/workspace/myshixun/

1.3. 实验思路

本次实验的核心任务是实现一个 Cache 模拟器，表面上看是对内存访问轨迹进行统计分析，但其本质是将计算机系统中 Cache 的工作机制，抽象成程序中的数据结构与算法流程。在实际编程实现前，我并没有急于编写代码，而是先围绕“如何准确模拟 Cache 行为”这一核心问题进行拆解，再逐步完成结构与功能实现，确保每一步都贴合 Cache 的底层原理。

整个实验思路可以归纳为四个核心问题：如何在程序中合理表示 Cache、如

如何将内存地址准确映射到 Cache、如何实现 LRU 替换策略，以及如何通过 trace 文件驱动整个模拟过程。下面结合我的设计思考与具体代码实现，详细说明每一步的思路与操作。

1.3.1 Cache 数据结构设计

在明确实验要求，掌握 Cache 基本原理后，我首先面临的问题是：在程序中应该如何表示 Cache，才能既符合其硬件原理，又便于后续的访问、修改等操作？

根据存储器层次结构的相关知识，Cache 并非简单的数组，而是由多个组 (Set) 组成，每个组内又包含若干个缓存行 (Line)。当访问内存地址时，会先将地址映射到某一个组，再在该组内查找对应的缓存行。因此，在设计程序结构时，我将 Cache 抽象为“组的集合”，并采用数组来存储这些组，这样既能贴合 Cache 的硬件结构，又能保证访问效率。

结合这一思路，我定义了如下结构体来表示 Cache：

```
typedef struct Cache {  
    int s;    // 组索引位数  
    int E;    // 每组行数  
    int b;    // 块偏移位数  
    int S;    // 组数 =  $2^s$   
    Cache_Set *sets;  
}
```

在这个结构体设计中，组数 S 并非固定值，而是根据输入的参数 s 动态计算得出，这样的设计能够保证程序适用于任意参数组合，不会局限于某种固定规模的 Cache，提升了程序的灵活性和通用性。

此外，使用数组 sets 存储所有组，是因为数组支持通过下标快速访问，这与 Cache 中“通过组索引定位组”的硬件行为完全一致，相当于在程序层面模拟了硬件中的组选择过程，确保了模拟的准确性。

进一步思考，每个组内部不仅需要存储缓存行的数据，还需要记录缓存行的使用顺序，以便实现 LRU 替换策略。因此，我又定义了组的结构体，具体如下：

```
typedef struct Cache_Set {  
    LRU_List *lru_list;  
    int capacity;  
    int size;  
}
```

这里引入了 LRU 链表来维护缓存行的访问顺序，为后续 LRU 策略的实现打

下基础。

1.3.2 LRU 机制实现

完成 Cache 结构设计后，下一个核心问题就是如何准确判断并替换“最近最少使用”的缓存行，也就是如何实现 LRU 策略。

在思考实现方式时，我没有选择简单的数组或时间戳记录法，而是决定采用双向链表来实现 LRU 策略。做出这个选择，主要是基于双向链表的三个优势：插入和删除操作效率高（时间复杂度为 $O(1)$ ）、可以方便地调整节点顺序、能够直观地体现“最近使用”和“最久未使用”的关系，完美契合 LRU 策略的核心需求。

首先，我定义了链表节点的结构体，用于存储每个缓存行的 Tag 和链表指针：

```
typedef struct LRU_Node {  
    unsigned long tag;  
    struct LRU_Node *prev;  
    struct LRU_Node *next;  
}
```

随后，定义了链表本身的结构体，通过头尾指针来管理整个链表：

```
typedef struct LRU_List {  
    LRU_Node *head;  
    LRU_Node *tail;  
}
```

基于这个双向链表结构，LRU 策略可以通过三个核心操作来实现。

第一，当缓存行命中时，更新其使用顺序，具体通过下面函数实现，这个函数的内部逻辑是：先将命中的节点从原来的位置移除，再将其插入到链表的头部。这样设计的原因是，命中的缓存行刚刚被访问过，属于最近使用的缓存行，应该放在链表头部，避免被优先替换。

```
static void move_to_head(LRU_List *list, LRU_Node *node) {  
    // 先从原位置断开  
    node->prev->next = node->next;  
    node->next->prev = node->prev;  
    // 插入到头部  
    node->next = list->head->next;  
    node->prev = list->head;  
    list->head->next->prev = node;  
    list->head->next = node;}  

```

第二，当缓存行缺失且组未滿时，插入新的缓存行，通过 `add_node(list, tag)` 添加新节点到头部。新加载到 Cache 中的数据，自然是最新访问的，因此直接将其插入到链表头部，符合 LRU 策略的逻辑。

第三，当缓存行缺失且组已滿时，驱逐最久未使用的缓存行，由于链表的尾节点始终代表最久未使用的缓存行，因此直接删除尾节点即可完成驱逐操作，无需额外判断，简化了代码逻辑。

1.3.3 地址解析与访问算法

确定了 Cache 和 LRU 的结构后，下一步就是实现 Cache 访问的核心逻辑，其中最关键的一步是如何将内存地址准确映射到具体的 Cache 组，以及如何处理访问过程中的命中、缺失和驱逐三种情况。

首先需要解决的问题是：如何将一个内存地址拆分为 Tag、Set Index 和 Block Offset 三部分，并通过计算定位到目标 Cache 组？

根据 Cache 的地址映射原理，我通过位运算完成了地址的解析，具体代码如下：

```
int set_idx = (addr >> cache->b) & ((1 << cache->s) - 1);
unsigned long tag = addr >> (cache->s + cache->b);
```

这段代码的逻辑的可以通俗地理解为：先将地址右移 `b` 位，去掉低位的 Block Offset（块内偏移）；再通过与运算提取中间的 `s` 位，得到 Set Index（组索引）；最后将地址右移 `s+b` 位，剩余的高位即为 Tag（标记），用于在组内查找具体的缓存行。

完成地址解析后，就可以通过组索引定位到目标组。

```
Cache_Set *set = &cache->sets[set_idx];
LRU_List *list = set->lrulist;
```

定位到目标组后，就进入了 Cache 访问的核心流程：首先在该组的 LRU 链表中查找是否存在与当前 Tag 匹配的节点。

```
LRU_Node *node = find_node(list, tag);
```

如果找到匹配的节点，说明发生了 Cache 命中，此时需要更新命中次数，并调整该节点在 LRU 链表中的位置。

```
hit_cnt++;
move_to_head(list, node);
```

这里需要特别注意，命中后必须更新 LRU 顺序，否则后续的替换策略会出现逻辑错误，无法准确判断“最近最少使用”的缓存行。

如果没有找到匹配的节点，说明发生了 **Cache** 缺失，此时需要更新缺失次数，并判断该组是否已满，进而决定是否需要驱逐缓存行，具体代码如下：

```
miss_cnt++;
if(set->size < set->capacity) {
    add_node(list, tag);
    set->size++;
} else {
    evict_cnt++;
    remove_tail(list);
    add_node(list, tag);
}
```

这段代码完整体现了 **Cache** 访问的三种情况：命中(hit)、缺失但组未滿(miss)、缺失且需要驱逐 (miss + eviction)，每一步操作都与真实 **Cache** 的工作行为一一对应，确保了模拟的准确性。

1.3.4 Trace 驱动与程序整体实现

解决了 **Cache** 结构、LRU 策略和地址访问逻辑后，我们需要将 **trace** 文件中的内存访问序列，转化为 **Cache** 的具体操作，驱动整个模拟过程的运行。

trace 文件中的每一行都代表一次内存访问，包含操作类型、内存地址和访问大小等信息。因此，我需要逐行读取 **trace** 文件中的内容，并对每一行进行解析

```
while (fscanf(fp, " %c %lx,%d", &op, &addr, &size) == 3)
```

解析出操作类型后，需要根据不同的操作类型执行对应的逻辑。其中，操作类型主要分为四种：I（指令访问）、L（加载操作）、S（存储操作）、M（修改操作），具体处理逻辑如下：

```
switch(op) {
    case 'I':
        break;
    case 'L':
    case 'S':
        access_cache(cache, addr);
        break;
    case 'M':
        access_cache(cache, addr);
        access_cache(cache, addr);
        break;}
}
```

这里需要重点说明的是对 M 操作的处理。M 操作并非一次内存访问，而是“读 + 写”两次访问，因此需要调用两次 `access_cache` 函数。如果只执行一次，会导致命中、缺失次数的统计错误，这也是我在调试过程中曾经遇到的问题，后来通过查阅实验要求和反复测试，修正了这一逻辑。

1.4. 实验结果和分析

完成 Cache 模拟器的编写后，我运行了实验提供的测试脚本，对模拟器的正确性进行验证，具体运行命令如下：

```
./test-csim
```

运行后得到的输出结果如下：

Points (s,E,b)	Your simulator			Reference simulator			
	Hits	Misses	Evicts	Hits	Misses	Evicts	
3 (1,1,1)	9	8	6	9	8	6	traces/yi2.trace
3 (4,2,4)	4	5	2	4	5	2	traces/yi.trace
3 (2,1,4)	2	3	1	2	3	1	traces/dave.trace
3 (2,1,3)	167	71	67	167	71	67	traces/trans.trace
3 (2,2,3)	201	37	29	201	37	29	traces/trans.trace
3 (2,4,3)	212	26	10	212	26	10	traces/trans.trace
3 (5,1,5)	231	7	0	231	7	0	traces/trans.trace
6 (5,1,5)	265189	21775	21743	265189	21775	21743	traces/long.trace

27

yi2.trace测试通过,
 yi1.trace测试通过,
 dave.trace测试通过,
 trace.trace测试通过,
 long.trace测试通过!

图 1-1 实验一运行结果

从输出结果可以看出，我编写的 Cache 模拟器与参考模拟器的输出完全一致，所有测试用例均通过，说明模拟器的实现是正确的。

2. 矩阵转置实验

2.1. 实验目的

本次实验建立在 **Cache** 工作原理的理论基础之上，以矩阵转置功能实现为载体，开展针对性的缓存性能优化实践，核心目标是在完成矩阵转置功能正确性的前提下，通过优化内存访问模式最大限度降低 **Cache** 缺失次数，提升程序执行效率。实验不仅要求实现基础转置功能，更需要深入理解存储器层次结构、程序局部性原理与缓存缺失产生的核心原因，将空间局部性与时间局部性理论真正应用到代码优化中。

本实验还要求在一定约束条件下完成优化，例如限制局部变量数量、禁止使用动态内存分配、不可修改原矩阵 **A** 等，这在一定程度上增加了实现难度，也更加贴近真实系统编程中的约束环境。通过本次实验，可以进一步提升我们对 **Cache** 优化的理解能力，以及在约束条件下进行程序优化的实践能力。

2.2. 实验环境

本实验同样在头歌远程实验环境中完成，环境配置与上一个实验相同

2.3. 实验思路

本次实验的核心任务是优化矩阵转置函数，使其在执行过程中尽量减少 **Cache miss**。从表面上看，这是一个简单的数组操作问题，但本质上是一个内存访问模式优化问题。在具体实现之前，我首先阅读了实验介绍，理解传统方法性能缺陷的原因，再逐步推导出优化策略，并针对不同规模的矩阵设计不同的实现方案。

最基础的矩阵转置实现如下：

```
for (i = 0; i < N; i++) {
    for (j = 0; j < M; j++) {
        B[j][i] = A[i][j];
    }
}
```

这段代码逻辑非常直观，但在 **Cache** 层面存在明显问题。其主要表现为：

一方面，矩阵 **A** 是按行访问的，即连续访问内存，这种访问方式具有良好的空间局部性，**Cache** 命中率较高；但另一方面，矩阵 **B** 是按列写入的，也就是说

写操作在内存中是跳跃进行的，这会导致 Cache 频繁加载新的块，从而产生大量 miss。

以 4*4 数组为例。

A 数组访问 A[0][0]，冷不命中，将块 1 装入 cache。

B 数组访问 B[0][0]，虽然 B[0][0]所映射的块 1 在 cache 中，但是标记位不同，造成冲突不命中，重新将数组 B 对应的块 1 装入 cache。

A 数组访问 A[0][1]，虽然 A[0][1] 所映射的块 1 在 cache 中，但是标记位不同，造成冲突不命中，重新将数组 A 对应的块 2 装入 cache。

B 数组访问 B[1][0]，虽然 B[1][0]所映射的块 2 在 cache 中，但是标记位不同，造成冲突不命中，重新将数组 B 对应的块 2 装入 cache。

分析上述可以看出 miss 过多的原因在于直接映射 Cache 中，A 和 B 中下标相同的位置很可能映射到同一 Cache 组，而由于 Tag 不同，会不断发生冲突替换。这种“反复加载—驱逐—再加载”的过程正是 miss 过多的主要原因。

基于这个原因，我们可以一次性访问同一个块中的多个元素，访问完以后便不再需要访问这个块了，从而可以大大地减少冲突不命中的数目。这里前 8 个元素在同一个块中，我们可以直接将这 8 个元素取出来，然后这 8 个元素所在的块便不再需要访问了。（即分块访问思想）

分块即是把矩阵分成子矩阵，子矩阵大小取决于缓存块大小、缓存大小、输入矩阵大小。

$$\begin{array}{cc}
 A = \begin{array}{|c|c|} \hline W & X \\ \hline Y & Z \\ \hline \end{array} & B = \begin{array}{|c|c|} \hline W' & Y' \\ \hline X' & Z' \\ \hline \end{array} \\
 \end{array}$$

在代码结构上，分块实现如下：

```

for (i = 0; i < N; i += block_size) {
    for (j = 0; j < M; j += block_size) {
        for (x = i; x < i + block_size; x++) {
            for (y = j; y < j + block_size; y++) {
                B[y][x] = A[x][y];
            }
        }
    }
}

```

通过这种方式，原本跨越整个矩阵的大范围访问，被限制在较小的局部区域内，从而显著提升 Cache 命中率。

在具体实现中，不同规模的矩阵对 Cache 的影响不同，因此需要分别设计优化策略。

(1) 32×32 矩阵

```
if (M == 32 && N == 32) {  
    for (i = 0; i < N; i += 8) {  
        for (j = 0; j < M; j += 8) {  
            for (x = i; x < i + 8; x++) {  
                for (y = j; y < j + 8; y++) {  
                    B[y][x] = A[x][y];  
                }  
            }  
        }  
    }  
}
```

在该情况下，Cache 块大小为 32 字节，即可以容纳 8 个 int 数据。因此选择 8×8 分块，可以使一个子块正好适配 Cache 结构。在块内部访问 A 时是连续的，而写入 B 也集中在较小范围内，显著减少了冲突 miss。

(2) 64×64 矩阵

```
else if (M == 64 && N == 64) {  
    for (i = 0; i < N; i += 4) {  
        for (j = 0; j < M; j += 4) {  
            for (x = i; x < i + 4; x++) {  
                for (y = j; y < j + 4; y++) {  
                    B[y][x] = A[x][y];  
                }  
            }  
        }  
    }  
}
```

对于 64×64 矩阵，如果仍然使用 8×8 分块，会导致多个块映射到相同 Cache 组，从而产生严重冲突。因此将块大小缩小为 4×4，降低每次访问的数据规模，减少 Cache 竞争。

需要说明的是，该实现虽然比直接转置有明显优化，但由于没有进一步利用 B 作为缓冲区进行数据重排，因此仍然存在较多 miss。

(3) 61×67 矩阵

```
else {
    for (i = 0; i < N; i += 8) {
        for (j = 0; j < M; j += 8) {
            for (x = i; x < i + 8 && x < N; x++) {
                for (y = j; y < j + 8 && y < M; y++) {
                    B[y][x] = A[x][y];
                }
            }
        }
    }
}
```

该矩阵为非规则尺寸，无法整齐分块，因此仍采用 8×8 分块，并在循环中增加边界判断：

$x < N \ \&\& \ y < M$

这样既能保持分块带来的局部性优势，又能避免越界访问。

2.4. 实验结果和分析

实验运行结果如下：

实际输出			
测试矩阵转置			
实验汇总:			
转置矩阵类型	得分该项分值是否有效		
32x32	6.9	8	343
64x64	1.2	8	1891
61x67	8.8	10	2118
合 计	16.9	26	
矩阵32x32转置优化完成,			
矩阵64x64转置优化完成,			
矩阵61x67转置优化完成!			

图 2-1 实验二运行结果

从结果可以看出：

对于 32×32 矩阵，采用 8×8 分块后，Cache miss 明显减少，性能较好，但由于未对对角线元素进行特殊处理，仍存在一定优化空间。

对于 64×64 矩阵，得分仅 1.2，缺失数 1891，接近评分阈值，优化效果有限。核心原因是仅采用基础 4×4 分块，未使用实验提示的利用 B 矩阵暂存数据、调整子块读写顺序等进阶优化手段，仍存在大量缓存冲突缺失，这也是本次实验可进一步提升的方向。

对于 61×67 矩阵，通过 8×8 分块 + 边界判断，缺失数 2118，得分 8.8，表现优秀，证明分块优化对规则方阵与非规则矩阵均具备良好的适用性。

总体而言，本次实现已经体现了分块优化的基本思想，并在大多数情况下取得了较好的效果。但也可以看出，仅靠简单分块仍难以达到最优性能，尤其是在 64×64 矩阵情况下，还需要更精细的访问控制策略，例如利用临时变量或分阶段搬运数据，以进一步降低 Cache 冲突。

losyi

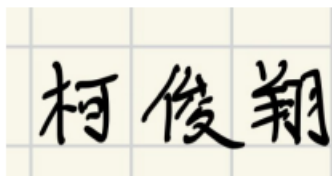
3. 总结和体会

通过这两个实验，我对 Cache 的工作原理和程序性能有了深刻的直观认识。Cache 模拟器让我理清了地址拆分、组相联、LRU 替换等底层机制；矩阵转置则让我体会到分块优化对提升空间局部性和命中率的关键作用。一个侧重底层实现，一个聚焦上层优化，两者相辅相成，帮助我系统理解了如何让程序高效利用 Cache。同时，工程能力也得到锻炼，比如位运算、动态内存分配、分块策略设计，以及通过缺失次数反推代码问题。总体而言，这是一次极具价值的系统级编程训练。

4. 对实验课程的建议

1. 对于 Cache 模拟器实验，建议扩充实验内容，分阶段实现不同映射方式和淘汰算法
2. 对于矩阵转置实验，建议改写实验介绍（知识点讲解），通过具体计算，动画的方式展示优化前后的访问轨迹，有助于学生理解。
3. 目前实验内容多年未变，可以考虑进行一定更新，覆盖更多理论课知识点，不仅限于缓存。

电子签名：

A handwritten signature in black ink on a yellow grid background. The signature consists of three characters: '柯', '俊', and '翔'.